Lösungsskizze Theorieteil — nur für Lehrer:in

> ***Hinweis:*** *Es handelt sich um Musterlösungen, nicht um die einzig zulässige Antwort. Eigenständige, sinnvoll begründete Abweichungen werden mit voller Punktzahl bewertet.*

Block A — UML & Architektur (7 P)

Frage 1 — Klassendiagramm zu Aufgabe 1 (3 P)

Erwartete Elemente (mind. einer pro Punkt):

```
                    «abstract»
                    Statement
                    + Execute(ctx: TableContext)
                          ▲
                          |
   ┌──────────┬──────────┬──────────┬──────────┬──────────┐
   |          |          |          |          |          |
TableStmt ColumnStmt RowStmt SummeStmt MittelStmt ZeigeStmt
                ▲
                | (Komposition über FORMEL)
                |
            «abstract»
              Expr
            + Eval(row): double
                ▲
                |
        ┌───────┴───────┐
        |               |
    AddSubExpr     MulDivExpr ──► Atom (Zahl | Spaltenref | Klammer)
```

Bewertung:

- Abstrakte `Statement`-Basisklasse mit `Execute` o. ä. (1 P)
- Mind. 4 konkrete Statement-Klassen mit Vererbungspfeilen (1 P)
- Operator-Precedence-Hierarchie für Formeln, mind. zwei Ebenen (1 P)

Akzeptabel sind alle gängigen Notationen (offene/geschlossene Pfeile, mit/ohne Sichtbarkeitsangaben), solange Vererbung klar von Aggregation/Komposition unterscheidbar ist.

Frage 2 — Sequenzdiagramm zu Aufgabe 2 (3 P)

Erwarteter Ablauf:

```
User       Client-UI    ClientTransfer    ServerTransfer    PermutationsTourSolver
 |            |              |                  |                        |
 | Klick      |              |                  |                        |
 |──"Beste──▶|              |                  |                        |
 |   Tour"    |              |                  |                        |
 |            | Send(MSG)    |                  |                        |
 |            |──────────────▶|                  |                        |
 |            |              | TourRequest XML |                        |
```

```
│            │              │──────────────▶│                      │
│            │              │               │ ReceiveMessage(MSG)  │
│            │              │               │──────────────────────▶│
│            │              │               │  Solve(selectedIds)  │
│            │              │               │◀- - - - - - - - - - - -│
│            │              │               │  (order, total)      │
│            │              │               │                      │
│            │              │               │ Send(TourAnswer)     │
│            │              │ TourAnswer XML │◀─────────────────────│
│            │              │◀──────────────│                      │
│            │ Receive(MSG) │               │                      │
│            │◀─────────────│               │                      │
│  Tour      │              │               │                      │
│◀-Anzeige──│              │               │                      │
```

Bewertung:

- Korrekte Aktivierungsbalken / Lebenslinien (1 P)
- Anfrage- und Antwortpfad vollständig, beide Transfers gezeigt (1 P)
- Solver wird vom Server aufgerufen (nicht direkt vom Client) (1 P)

Frage 3 — Architekturvergleich (1 P)

Erwartung: Mind. 2 Vorteile + 1 Nachteil. Beispiele:

Vorteile (je ½ P):

- Daten an einem Ort: Es gibt nur eine Datenquelle, mehrere Clients greifen konsistent darauf zu.
- Rechenintensive Operationen am Server: Schwacher Client (z. B. Tablet) reicht.
- Trennung von Logik (Algorithmen) und Visualisierung erlaubt unabhängige Weiterentwicklung beider Seiten.
- Mehrbenutzerfähigkeit: mehrere Clients können parallel zugreifen.

Nachteile (je ½ P):

- Höherer Implementierungsaufwand (Serialisierung, Protokoll, Threading).
- Bei Netzwerkausfall ist die Anwendung unbenutzbar.
- Latenz: jede Aktion benötigt einen Roundtrip.

Bewertung: 2 sinnvolle Vorteile **und** 1 sinnvoller Nachteil = volle Punktzahl.

Block B — Algorithmen & Komplexität (7 P)

Frage 4 — Dijkstra-Trockenlauf (2 P)

Graph-Kanten: A–B(4), A–D(2), B–C(5), B–D(1), D–C(3), D–E(7), C–E(6) Start: **A**

Iteration	Finalisiert	dist(A)	dist(B)	dist(C)	dist(D)	dist(E)	Vorgänger (B,C,D,E)
0	–	0	∞	∞	∞	∞	–
1	A	0	4	∞	2	∞	(A, –, A, –)
2	D	0	3	5	2	9	(D, D, A, D)
3	B	0	3	5	2	9	(D, D, A, D)

4	C	0	3	5	2	9	(D, D, A, D)
5	E	0	3	5	2	9	(D, D, A, D)

Wichtige Übergänge:

- Iter. 2: über D wird B billiger (2+1=3) und C erreichbar (2+3=5), E erreichbar (2+7=9).
- Iter. 4: C–E würde 5+6=11 ergeben, schlechter als 9 → keine Aktualisierung.

Bewertung:

- Korrekte Reihenfolge der finalisierten Knoten (1 P)
- Korrekte Endwerte aller Distanzen und Vorgänger (1 P)

Frage 5 — K-Means (2 P)

Erwartete Punkte (je ½ P, max. 2):

1. **Initialisierung:** k Zentroide werden zufällig (oder mit k-means++) gesetzt.
2. **Zuordnung:** Jeder Datenpunkt wird dem nächstgelegenen Zentroid zugewiesen (Distanzmetrik, meist euklidisch).
3. **Aktualisierung:** Jedes Zentroid wird auf den Schwerpunkt seiner zugewiesenen Punkte verschoben.
4. **Abbruch:** wenn sich keine Zuordnung mehr ändert oder eine maximale Iterationszahl erreicht ist.
5. **Schwächen** (eine genügt): Anzahl k muss vorher festgelegt werden; Lösung hängt von Initialisierung ab; nur konvexe, runde Cluster werden gut erkannt; Ausreißer verzerren Zentroide.

Frage 6 — Permutationen (1 P)

- N = 6 → **720** Permutationen (½ P)
- N = 10 → **3 628 800** Permutationen (¼ P)
- Begründung: Bei 9! oder 10! wären es schon Millionen Pfade, was selbst bei wenigen Mikrosekunden pro Pfad mehrere Sekunden Rechenzeit benötigt — das wird in einer Live-UI unangenehm. Limit auf 8 (40 320 Permutationen) hält die Antwort unter ~100 ms. (¼ P)

Frage 7 — Pseudocode rekursiver Permutationsgenerator (2 P)

```
funktion Permutiere(rest, prefix):
    falls rest leer:
        gib prefix aus
        return

    für jedes element in rest:
        neuesRest    = rest ohne element
        neuesPrefix  = prefix + element
        Permutiere(neuesRest, neuesPrefix)

# Aufruf für alle Permutationen:
Permutiere(eingabeliste, leereListe)

# Mit fixiertem Startpunkt (Aufgabe 2):
Permutiere(eingabe ohne erstes, [eingabe[0]])
```

Bewertung:

- Rekursive Struktur klar erkennbar (1 P)

- Korrekte Behandlung des Basisfalls und Aufbau des Prefixes (1 P)

Akzeptabel sind auch iterative Lösungen (Heap-Algorithmus, Lexikografisch-nächste-Permutation), solange sie nachvollziehbar sind und sich auf den Aufgaben-Use-Case beziehen.

Block C — Parser & Grammatik (4 P)

Frage 8 — EBNF (2 P)

```
ColumnStmt   = "COLUMN" , Bezeichner , ColumnType ;
ColumnType   = "TEXT"
             | "ZAHL"
             | "FORMEL" , Ausdruck ;

Ausdruck     = Term , { ("+" | "-") , Term } ;
Term         = Faktor , { ("*" | "/") , Faktor } ;
Faktor       = Zahl
             | Bezeichner
             | "(" , Ausdruck , ")" ;
```

Bewertung:

- Korrekte Verwendung von `=`, `|`, `,`, `[ ]`, `{ }` (1 P)
- `FORMEL`-Variante mit echtem rekursivem Ausdruck (1 P)

Auch zulässig: kompaktere Schreibweise mit optionalen Klammern oder `?`-Operator. Die *Tatsache*, dass Operator-Precedence durch verschachtelte Regeln (Term/Faktor) ausgedrückt wird, ist der zentrale Bewertungspunkt.

Frage 9 — Tokens (1 P)

Eingabe: `COLUMN Gesamt FORMEL (Menge + 1) * Preis`

#	Typ	Wert
1	KEYWORD	COLUMN
2	IDENT	Gesamt
3	KEYWORD	FORMEL
4	LPAR	(
5	IDENT	Menge
6	OP_PLUS	+
7	NUMBER	1
8	RPAR	)
9	OP_MUL	*
10	IDENT	Preis

Bewertung: Reihenfolge und Typkategorien korrekt → 1 P. Geringe Abweichungen in der Typbenennung (`KW` statt `KEYWORD` , `IDENTIFIER` statt `IDENT`) sind akzeptabel.

Frage 10 — Operator-Precedence (1 P)

Beispiel: `2 + 3 * 4`

Bei naiver Linksauswertung:

- `(2 + 3) * 4 = 5 * 4 = 20`

Mathematisch korrekt:

- `2 + (3 * 4) = 2 + 12 = 14`

Begründung: `*` bindet stärker als `+` . Eine korrekte Implementierung muss das in der Parser-Hierarchie (Term/Faktor bzw. AddSub/MulDiv) berücksichtigen.

Bewertung: Beispiel + zwei Ergebnisse + Begründung der Precedence → 1 P.

Block D — Netzwerk & Datenformate (2 P)

Frage 11 — TCP vs. UDP (1 P)

- **TCP**: verbindungsorientiert, garantiert Reihenfolge und Vollständigkeit, fehlertolerant durch Retransmits, höherer Overhead.
- **UDP**: verbindungslos, keine Garantien, geringer Overhead, schneller.
- **Begründung TourPlaner**: Anfragen sind klein, aber müssen vollständig und in Reihenfolge ankommen (sonst kann der XML-Parser die Nachricht nicht reassemblieren). Der Overhead spielt keine Rolle, weil pro Klick nur eine Nachricht fließt → TCP ist die natürliche Wahl.

Frage 12 — Nachrichtenformat & Serialisierung (1 P)

Erwartete Felder einer einheitlichen `MSG` -Klasse (½ P):

Feld	Wofür
`Type` (Enum)	Diskriminator: PathRequest/PathAnswer/ClusterRequest/...
`FromId`, `ToId`	Eingaben für Dijkstra
`K`	Eingabe für KMeans
`SelectedIds`	Eingabe für Permutationen
`PathIds`, `Distance`	Antwort Dijkstra
`Clusters` (Liste {PoiId, Cluster})	Antwort KMeans
`OrderedIds`, `Distance`	Antwort Permutationen
`Error` (string?)	Fehlerantwort

Serialisierung (½ P):

- **XML**: + lesbar, von .NET out-of-the-box; − verbose, langsamer.
- **JSON**: + kompakter, weit verbreitet; − zusätzliche Library nötig (Newtonsoft / System.Text.Json).

- **Binary**: + sehr klein und schnell; − schwer debugbar, brüchig bei Schema-Änderungen.

Bewertung: Sinnvolle Felder mit Begründung + Serialisierungsformat mit beidseitig korrekter Argumentation → 1 P.

Punkteübersicht Theorieteil

Block	Frage	max. P
A	1 Klassendiagramm	3
A	2 Sequenzdiagramm	3
A	3 Architekturvergleich	1
B	4 Dijkstra-Trockenlauf	2
B	5 K-Means	2
B	6 Permutationen-Komplexität	1
B	7 Pseudocode	2
C	8 EBNF	2
C	9 Tokenisierung	1
C	10 Operator-Precedence	1
D	11 TCP vs. UDP	1
D	12 Nachrichtenformat & Serialisierung	1
Summe		**20**